

Project Documentation



Project Title:	Health Monitoring System
Project Members:	23-NTU-CS-1247 (Arfa Faheem) 23-NTU-CS-1245 (Amna Sajid)
Course Code:	CSE-3080 (IOT project)
Class:	BS-AI (5th)
Group Name:	Group 7
Submitted To:	Mr. Nasir Mahmood

Health Monitoring System(HMS)

Introduction:

This Health Monitoring System(HMS) is designed to monitor vital health parameters such as heart rate and blood oxygen level (SpO₂) using IoT technology. The system uses ESP32 along with MAX30100 pulse oximeter sensor to collect real-time health data and display it locally on an OLED display as well as on the Blynk mobile application and dashboard, making it useful for home based health monitoring.

Problem Statement:

Regular monitoring of heart rate and blood oxygen level is important for maintaining good health, but frequent hospital visits and costly medical devices make it difficult for home-based patients. There is a need for a low-cost, real-time, and remote health monitoring system that can continuously track vital signs and alert users during abnormal conditions. This will also help in early diagnosis of a health risk. This project develops an IoT-based Health Monitoring System using ESP32 and MAX30100 to monitor, display, and transmit health data efficiently and also alert the user.

Objectives:

- To measure heart rate using MAX30100 sensor
- To monitor blood oxygen level(SpO₂)
- To display sensor values on OLED display
- To send real time data to Blynk cloud and mobile app
- To generate an alert by using buzzer when heart rate exceeds limit and also sends email message.

Libraries Used:

```
#include <Wire.h>
#include <MAX30100_PulseOximeter.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
```

Hardware Components:

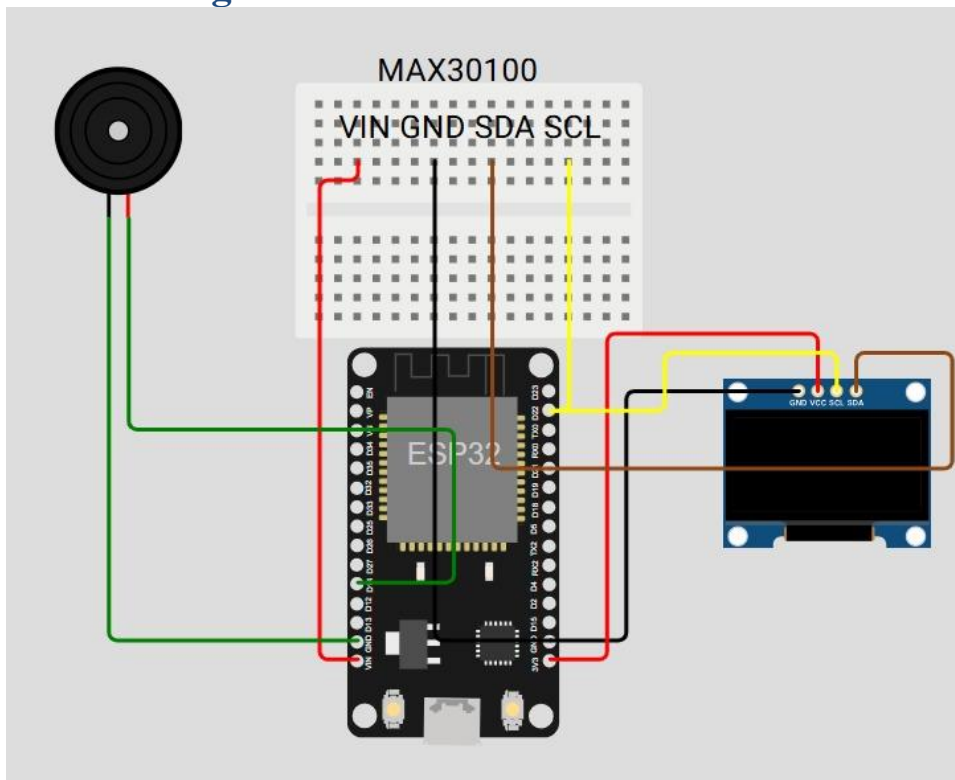
- ESP32 – Used as main microcontroller
- MAX30100 – Used to measure heart rate and SpO₂
- OLED Display – Used to display sensor values

- Buzzer – Used for alert indication
- Jumper wires

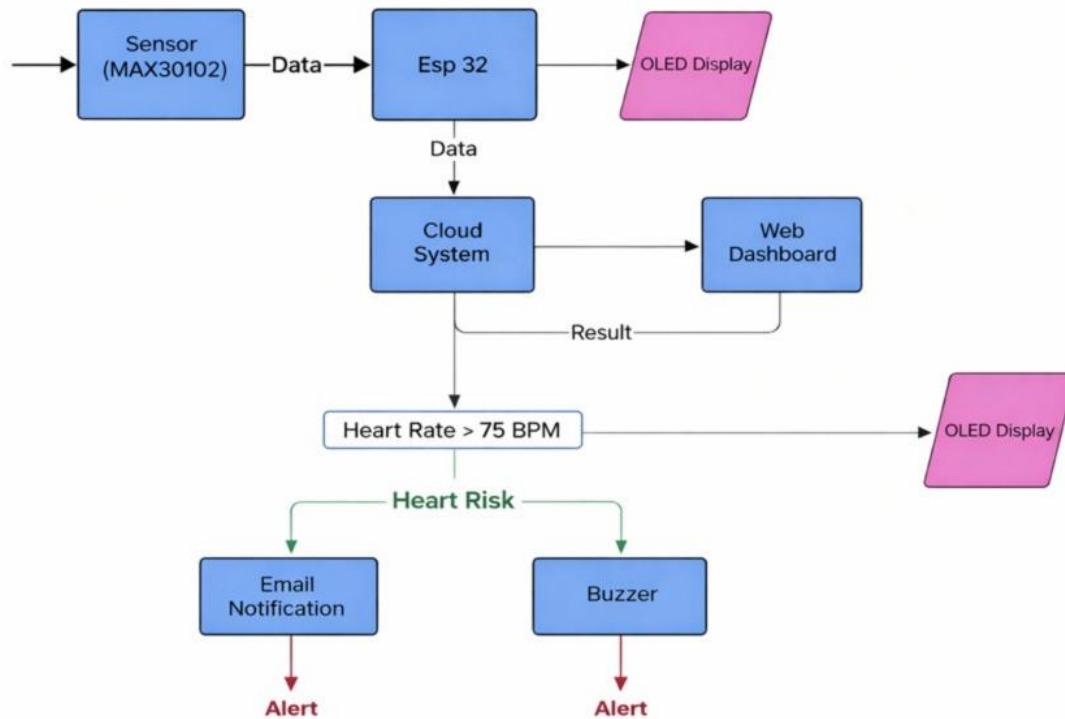
Software components:

- PlatformIO (VSCode) – IDE with advanced features and library management.
- Blynk Cloud – IoT platform for real-time data visualization and remote device control.
- Blynk Email Event – For sending email notifications and alerts.
- Blynk Dashboard – For showing sensor's readings and app notification.
- WiFi Libraries – For network connectivity on microcontrollers.

Circuit Diagram:



System Architecture (Flow chart):



Wiring Connections (Pins):

MAX30100 Pulse Oximeter

- SDA → ESP32 GPIO 21
- SCL → ESP32 GPIO 22
- VCC → 3.3V
- GND → GND

OLED Display (SSD1306 – I²C)

- SDA → ESP32 GPIO 21
- SCL → ESP32 GPIO 22
- VCC → 3.3V
- GND → GND

Buzzer

- Positive (+) → ESP32 GPIO 14
- Negative (–) → GND

ESP32

- Powered via USB

- Wi-Fi used for Blynk cloud connectivity

System Working:

- The ESP32 connects to WiFi and Blynk Cloud using an authentication token.
- The MAX30100 sensor continuously measures heart rate (HR) and SpO₂ values.
- These values are updated every 3 seconds and displayed on the OLED screen.
- Data is simultaneously sent to the Blynk application for real-time monitoring.
- If the heart rate exceeds 75 bpm, the buzzer is activated and an email notification is sent to alert about heart risk. (Normal value is 95 bpm but for testing, 75 bpm is used)
- The ESP32 stores recent sensor readings in memory to track trends over time.
- The system can differentiate between normal and abnormal SpO₂ levels and triggers alerts accordingly.
- Users can monitor their health remotely using the Blynk mobile app.
- The system logs data for future analysis and can generate reports on health conditions.
- All sensor readings are continuously monitored to provide immediate feedback.
- Alerts and notifications ensure timely action in case of critical health conditions.
- An email is also sent to the device owner to check their health status.

Conclusion:

The system shows heart rate in BPM and SpO₂ in percentage correctly. These values can be seen on the OLED display and also on the Blynk mobile app. When the heart rate goes above the safe limit, the buzzer turns on and an alert is sent to email to alert about heart risk. This system helps in real-time monitoring of health conditions. Overall, it is simple, low cost, and easy to use.

Limitations:

The MAX30100 pulse oximeter sensor shows limited compatibility when used side by side with other sensors in the same ESP32. During implementation, it was observed that when the MAX30100 was used together with the DS18B20 body temperature sensor, the MAX30100 failed to provide accurate heart rate and SpO₂ readings, while the DS18B20 continued to function normally.

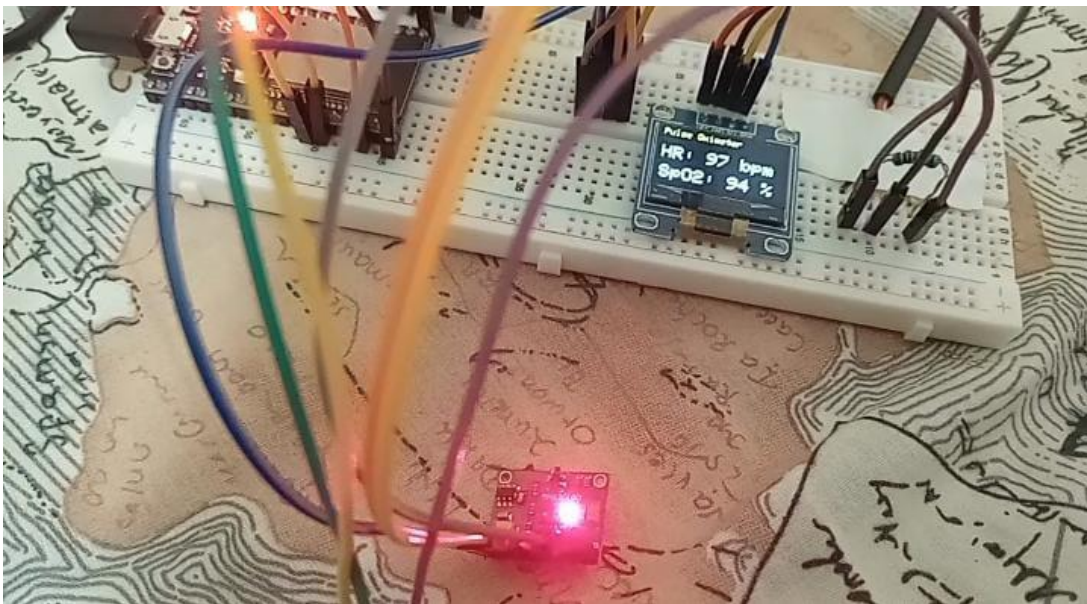
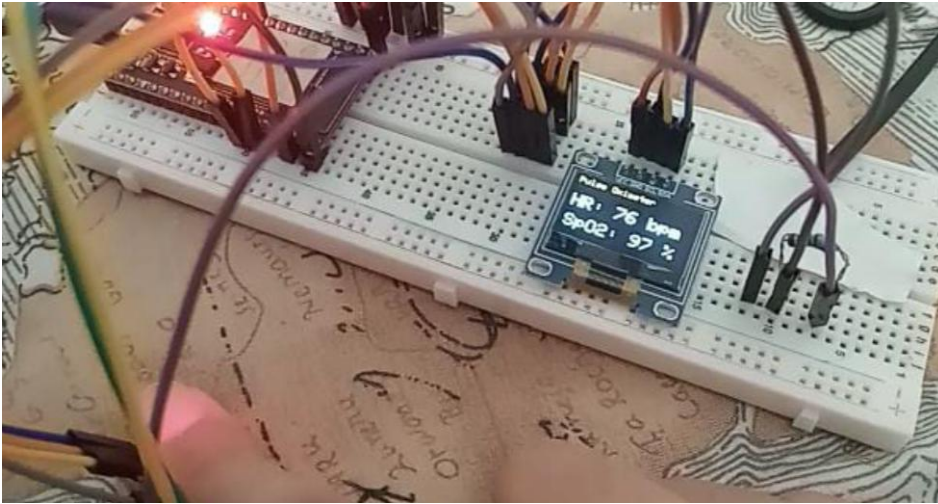
This issue occurs due to timing and resource conflicts, as the MAX30100 requires continuous I²C communication and frequent data updates, whereas the DS18B20 relies on precise timing for One Wire communication. Managing both sensors simultaneously on the ESP32 caused synchronization issues, leading to unreliable MAX30100 readings.

This limitation has been addressed in the MAX30102 sensor, which offers improved internal timing control and better stability when used in multi-sensor environments.

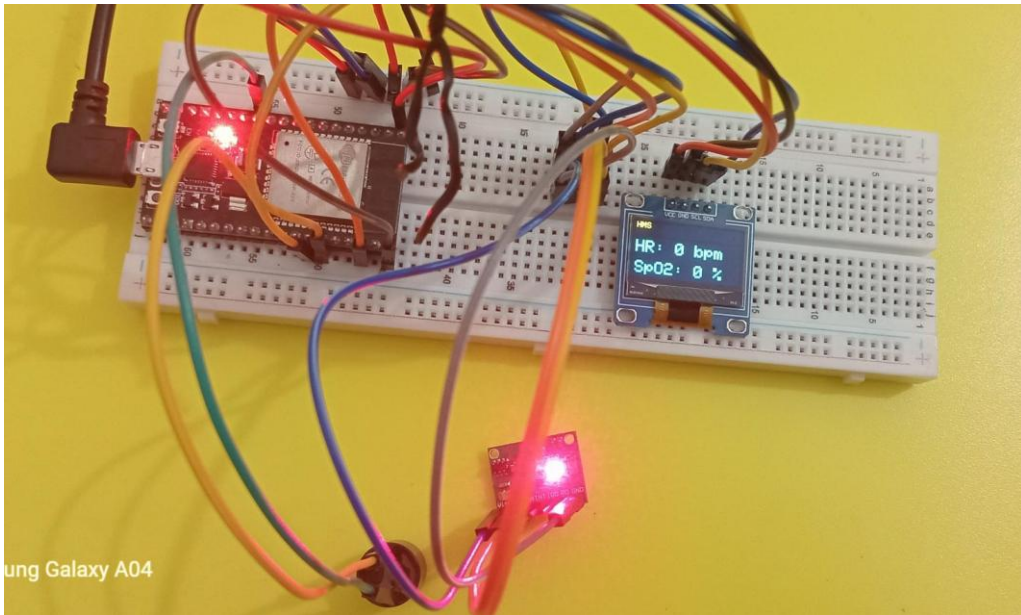
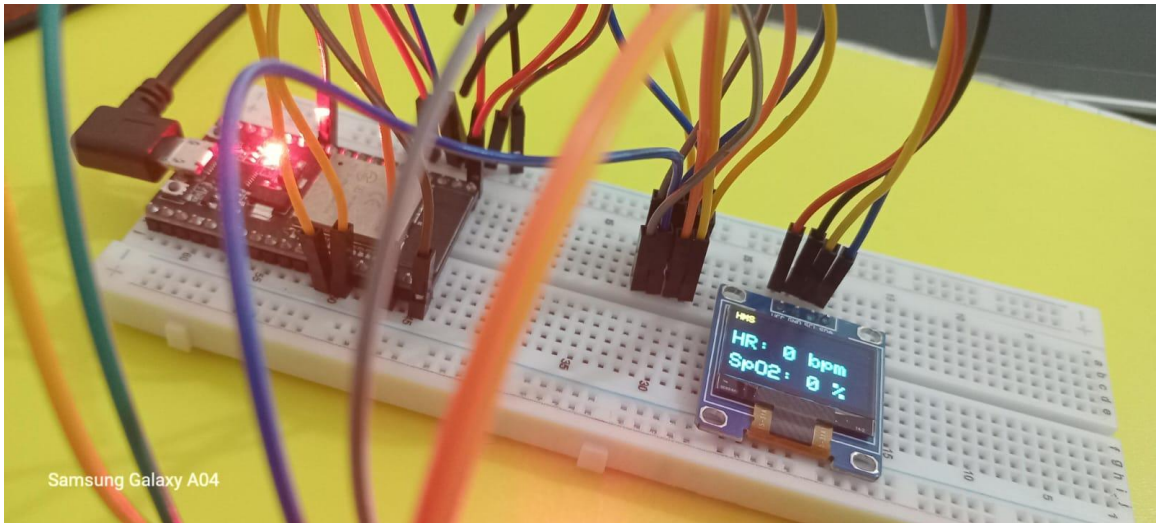
However, the MAX30102 was not easily available in the local market at the time of development, so the MAX30100 was used instead. As the project was about health monitoring, we use MAX30100 with alert system.

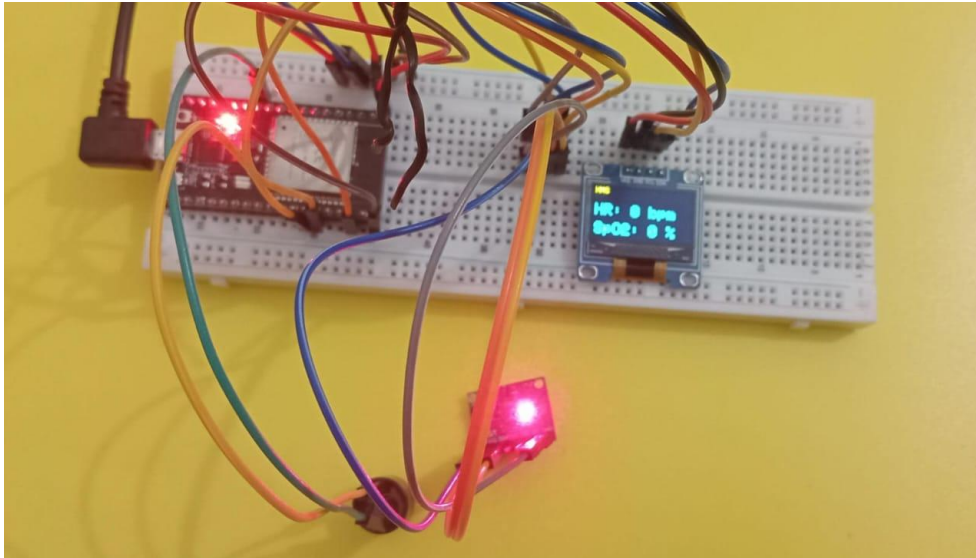
Screenshots

Project Working (hardware):



Hardware Wiring connection:





Alert on Gmail:



Blynk <robot@blynk.cloud>
to me ▾

10:39 AM (6 minutes ago)

hr alert

⚠ High Heart Rate Detected!
HR: 75 bpm

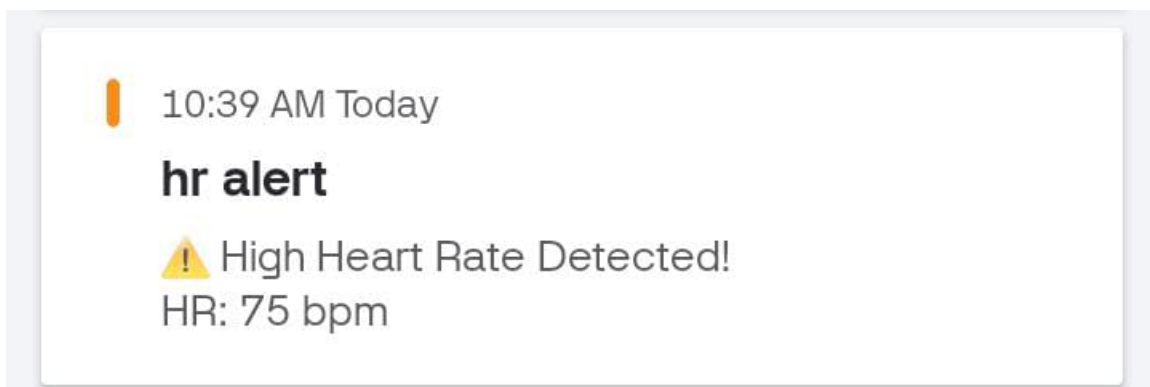
[Open in the app](#) | [Mute notifications](#)

--

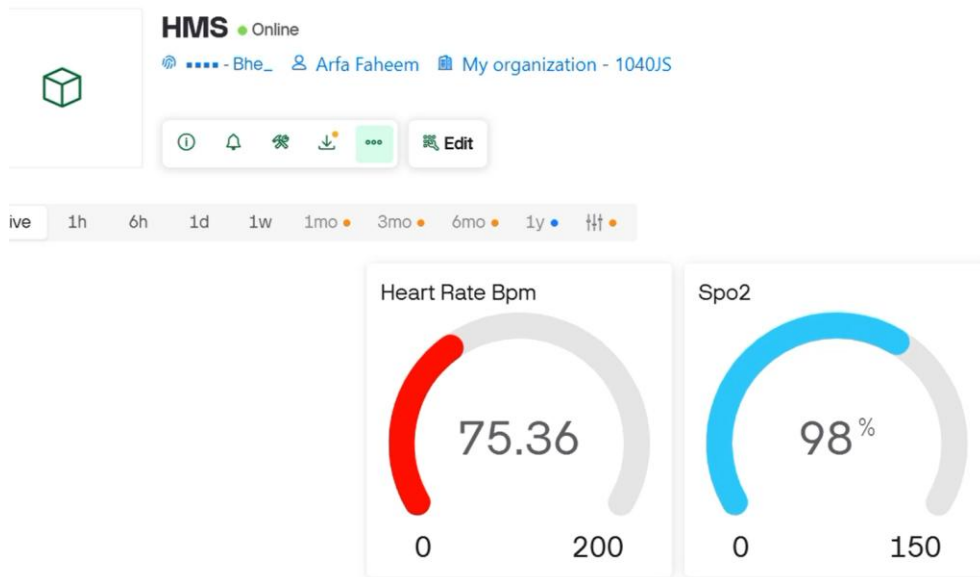
Date: Sunday, December 21, 2025, 10:39:47 AM Pakistan Standard Time

...

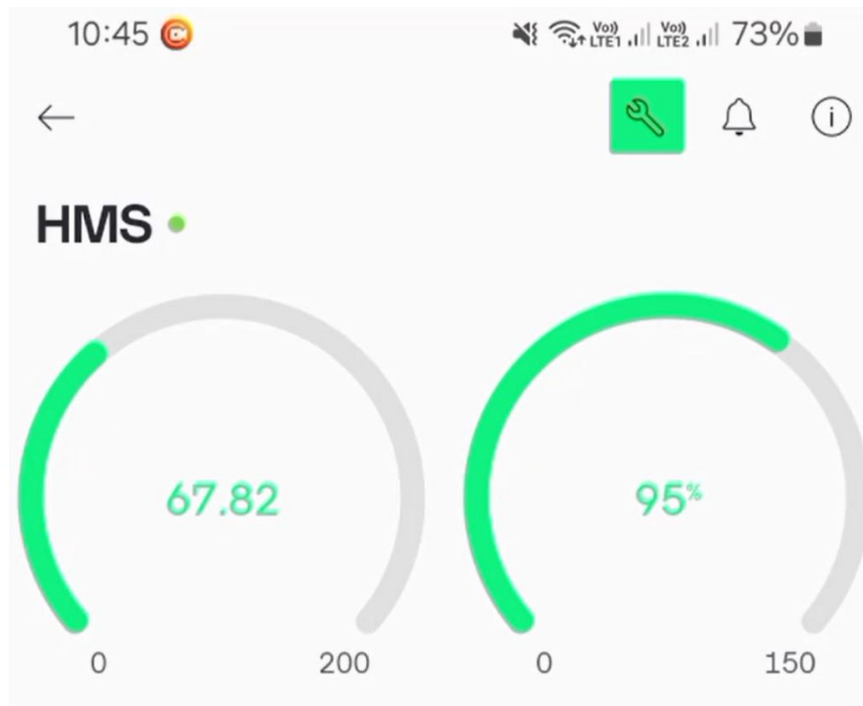
Alert on mobile app:



Blynk Dashboard:



Blynk App:



Code

```
#define BLYNK_TEMPLATE_ID "TMPL6lWpKA16D" //identifies blynk project with  
template id
```

```
#define BLYNK_TEMPLATE_NAME "Health Monitoring System" //project name on blynk cloud
```

```
#define BLYNK_AUTH_TOKEN "zmtY11y0b9sfY8m7SzdOs2cnrgcVBhe_" //unique key to connect esp32 to blynk dashboard
```

```
#define BLYNK_PRINT Serial //prints blynk connection messages on serial monitor
```

```
//libraries used throughout the project
```

```
#include <Wire.h> //for I2C communication
```

```
#include <OneWire.h>
```

```
#include <MAX30100_PulseOximeter.h> //for heart rate and SpO2 sensor
```

```
// #include <DallasTemperature.h> //for body temperature sensor
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h> //for OLED display
```

```
#include <WiFi.h>
```

```
#include <BlynkSimpleEsp32.h> //for wifi and blynk cloud connection
```

```
//Wifi credentials
```

```
char ssid[]="WirelessNet";
```

```
char pass[]="eeeeeeee";
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
#define OLED_RESET -1
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

```
#define REPORTING_PERIOD_MS 3000
```

```
PulseOximeter pox; // Creates sensor object pox for MAX30100(heart rate and SpO2)
```

```
uint32_t tsLastReport = 0; //for updating data every second
```

```
// Thresholds
```

```
#define HR_MAX 75 // Maximum heart rate threshold for alert
```

```
#define BUZZER_PIN 14
```

```
int melody[] = {523, 587, 659, 698}; // C5, D5, E5, F5
```

```
int noteDurations[] = {200, 200, 200, 400}; // ms
```

```
int melodyIndex = 0;
```

```
unsigned long noteStartTime = 0;
```

```
bool playingMelody = false;
```

```
// bool hrAlertSent = false;
```

```
unsigned long lastAlertTime = 0;
```

```
const unsigned long ALERT_INTERVAL = 30000; // 30 sec between repeated alerts
```

```
//Called when heartbeat is detected and prints message on serial monitor, callback function
```

```
void onBeatDetected() {
```

```
    Serial.println("Beat Detected!");
```

```
}
```

```
void setup() {  
  Serial.begin(115200);  
  
  //Blynk Initialization  
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass); //connect esp32 to wifi and blynk cloud  
  pinMode(BUZZER_PIN, OUTPUT);  
  digitalWrite(BUZZER_PIN, LOW); // buzzer off initially  
  
  //OLED Initialization  
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {  
    Serial.println(F("SSD1306 allocation failed"));  
    while (true);  
  }  
  
  display.clearDisplay();  
  display.setTextSize(1);  
  display.setTextColor(SSD1306_WHITE);  
  display.setCursor(0, 0);  
  display.println("Initializing...");  
  display.display();  
  
  //MAX30100 Initialization  
  if (!pox.begin()) { //checks if sensor (pox is sensor's object) is connected properly  
    Serial.println("MAX30100 FAILED");  
    display.setCursor(0, 10);  
    display.println("MAX30100 FAILED");  
  }  
}
```

```

    display.display();

    while (true);

} else {

    Serial.println("MAX30100 SUCCESS");

    display.setCursor(0, 10);

    display.println("MAX30100 SUCCESS");

    display.display();

}

    pox.setIRLedCurrent(MAX30100_LED_CURR_11MA); //setting the current for the IR LED
inside the MAX30100 pulse oximeter sensor

    pox.setOnBeatDetectedCallback(onBeatDetected); //if connected then links the callback
function to heartbeat detection

}

void loop() {

    Blynk.run();    // keeps Blynk cloud connection alive

    pox.update();    // IMPORTANT, must run continuously and read sensor data

    if (millis() - tsLastReport > REPORTING_PERIOD_MS) { // Updates data every 3 sec

        float hr = pox.getHeartRate(); // reads heart rate value

        float spo2 = pox.getSpO2();    // reads SpO2 value

        // Print data on Serial Monitor

        Serial.print("Heart rate: ");

        Serial.print(hr);

        Serial.print(" bpm / SpO2: ");

```



```
Serial.print(spo2);

Serial.println(" %");


// Send sensors' data to Blynk
Blynk.virtualWrite(V0, hr);
Blynk.virtualWrite(V1, spo2);


// Sensor values display on OLED
display.clearDisplay();
display.setCursor(0, 0);
display.setTextSize(1);
display.println("HMS");
display.setTextSize(2);
display.setCursor(0, 20);
display.print("HR: ");
display.print((int)hr);
display.println(" bpm");
display.setCursor(0, 45);
display.print("SpO2: ");
display.print((int)spo2);
display.println(" %");
display.display();


//for buzzer and email alert on high heart rate
unsigned long currentMillis = millis();
```

```

// HR Alert

if (hr > HR_MAX) {

    // Play melody

    if (!playingMelody) {

        noteStartTime = currentMillis;

        playingMelody = true;

    }

    if (playingMelody) {

        if (currentMillis - noteStartTime >= noteDurations[melodyIndex]) {

            tone(BUZZER_PIN, melody[melodyIndex], noteDurations[melodyIndex]);

            noteStartTime = currentMillis;

            melodyIndex++;

            if (melodyIndex >= sizeof(melody)/sizeof(melody[0])) {

                melodyIndex = 0;

                playingMelody = false;

            }

        }

    }

}

// Send Blynk event email every ALERT_INTERVAL

if (millis() - lastAlertTime >= ALERT_INTERVAL) {

    Blynk.logEvent(

        "hr_alert",

        " 🚨 High Heart Rate Detected!\nHR: " + String((int)hr) + " bpm"

    );

    lastAlertTime = millis();

```

```
}
```

```
} else {
```

```
    noTone(BUZZER_PIN); //stop buzzer
```

```
    playingMelody = false;
```

```
    melodyIndex = 0;
```

```
}
```

```
    tsLastReport = currentMillis;
```

```
}
```

```
}
```